CAR PRICE PREDICTION

CodeAlpha Machine Learning Project

A simple, beginner-friendly guide from start to finish

What you will learn

- Load and inspect a car dataset
- Clean data and create visualizations
- Train a machine-learning regression model
- Evaluate predictions using MAE, RMSE, and R^2
- Save the model and prepare a professional GitHub submission

Designed for a first-time learner. Run the matching Jupyter notebook cell by cell.

1. What is this project?

Simple explanation

A used-car buyer or seller may want an estimated price. We use examples of cars and their known selling prices to teach a computer to predict a likely price from car details.

Project type

This is **supervised machine learning** and specifically a **regression** problem, because the answer is a number rather than a category.

Important limitation

A prediction is not a guaranteed market value. Actual prices depend on location, condition, service history, accidents, demand, and date.

2. What you need

Tools

A computer with internet access for the default dataset, or Google Colab in a browser. The notebook uses Python, pandas, NumPy, Matplotlib, Seaborn, scikit-learn, and joblib.

Files

Use the companion file **CodeAlpha_Car_Price_Prediction.ipynb**. The notebook downloads a public teaching dataset; if download fails, it asks you to upload a compatible CSV.

3. Start in Google Colab

Step 1

Open Google Colab.

Step 2

Choose **File** → **Upload notebook** and select the provided `.ipynb` file.

Step 3

Run the first code cell with the play button. Continue from top to bottom.

Step 4

If a cell shows an error, read the full message. Check internet access, column names, and that you did not skip an earlier cell.

4. Understand the dataset

Target column

Selling_Price is the value the model learns to predict.

Possible input columns

Year (car year), **Present_Price** (showroom price), **Kms_Driven** (distance driven), **Fuel_Type**, **Seller_Type**, **Transmission**, and **Owner**. The teaching dataset may use price units such as lakhs; check the source and preserve those units.

Why inspect first?

Looking at shape, sample rows, data types, missing values, and duplicate rows helps you catch mistakes before modeling.

5. Data cleaning

What the notebook does

It standardizes column names, removes exact duplicate rows, converts expected numeric fields, drops rows without a target price, and checks that the target column exists.

Why not remove every unusual value?

An unusual value may be a real car. Investigate outliers using context before deleting data.

6. Explore with charts

Price distribution

A histogram shows how often different price ranges appear.

Price vs year

A scatter plot helps show how selling prices vary across model years.

Price vs kilometers

A scatter plot helps inspect the relationship between distance driven and price.

Price by fuel type

A box plot compares the distribution of prices across fuel categories.

Remember

Charts show patterns and associations. They do not by themselves prove cause and effect.

7. Train the model

Features and target

The input table `X` contains car details; `y` contains selling prices. The teaching notebook excludes `Car_Name` because it can create many unique categories in a small dataset.

Train/test split

The notebook keeps 20% of rows for testing and trains on the remaining 80%. The test set helps check performance on rows not used to fit the model.

Missing values

Numeric missing values are filled with the median. Text missing values are filled with the most frequent category.

Text features

One-hot encoding converts text categories into indicator columns so the model can use them.

Random Forest

The model trains many decision trees and averages their outputs. It is a useful baseline for tabular data.

8. Evaluate honestly

MAE

Mean Absolute Error is the average absolute gap between actual and predicted prices. Lower is better.

RMSE

Root Mean Squared Error gives more weight to large errors. Lower is better.

R^2

R-squared compares the model with a baseline that predicts the test-set mean. Values closer to 1 are generally better; a negative value can mean poor performance.

Interpretation

All error values use the same units as the target. Do not claim a high score without showing the test method, dataset limitations, and actual metric values.

9. Make a prediction and save

Example prediction

The notebook takes one row from the test set and prints the model's estimated selling price. Using a test row avoids needing to invent input columns.

Save the model

The notebook uses `joblib.dump()` to save the whole pipeline. It includes preprocessing as well as the trained model, which helps keep future predictions consistent.

Safety

Only load model files you trust.

10. Make it more professional

Ideas to try next

Compare Random Forest with Linear Regression, Decision Tree, and Gradient Boosting. Use cross-validation and tune model settings. Create a `Car_Age` feature using a clearly documented reference year. Test errors by price range and fuel type. Use a newer and more representative dataset if possible.

Optional app

Build a simple Streamlit form where a user enters car details and sees an estimated price.

Professional standard

Add a README, requirements file, data source, screenshots, metric results, and limitations. Test the notebook in a fresh session.

11. GitHub and internship submission

Repository

Create a repository named **CodeAlpha_CarPricePrediction**, unless your internship instructions require a different exact name.

Files

Upload the notebook, a README, and requirements.txt. The saved `.joblib` model is optional; check size and dataset restrictions before uploading.

Suggested requirements.txt

```
pandas
numpy
matplotlib
seaborn
scikit-learn
joblib
jupyter
```

README sections

Project overview, problem statement, dataset source, features, steps to run, evaluation metrics, results, limitations, future improvements, and screenshots.

LinkedIn video outline

Introduce the problem; show the dataset; explain cleaning and charts; describe the model; show metrics and a prediction; state limitations; share the repository link. Then submit through the internship form as directed by your program.

12. Interview practice

Q: What is the objective?

A: To estimate a used car's selling price from its features.

Q: Is this classification or regression?

A: Regression, because the target is a continuous numeric value.

Q: What is the target variable?

A: `Selling_Price`.

Q: Why split training and testing data?

A: To estimate how the model performs on unseen rows.

Q: What is data leakage?

A: When information that would not be available at prediction time improperly helps training or evaluation.

Q: Why use one-hot encoding?

A: To represent text categories as numeric indicators.

Q: What does MAE mean?

A: Average absolute difference between actual and predicted values.

Q: Why use Random Forest?

A: It combines multiple decision trees and can learn non-linear patterns.

Q: What are the limitations?

A: Data may be small or old, and price depends on factors missing from the dataset.

Q: How can it be improved?

A: Use better data, cross-validation, tuning, meaningful features, and subgroup error analysis.

Q: Can it guarantee the true price?

A: No. It gives an estimate based on the training data.

13. Final checklist

Before submission

■ Notebook runs top to bottom. ■ You understand the target and price units. ■ Charts have titles and labels. ■ MAE, RMSE, and R^2 are reported. ■ You show a sample prediction. ■ The model saves and reloads. ■ GitHub README describes limitations. ■ LinkedIn video and submission form are completed.